Retrofit Radar

Predicting building energy use to decide which buildings to retrofit first

---

Chua Guan Yu · Chay Ze Huan · Li Shou Xiang · Liu Xiang Zhe

Learn To Hack: MATLAB Edition 2026 · NTU · Built in MATLAB

Which buildings should be fixed first?

- Buildings use about a third of the world’s energy and energy-related CO_2.
- Retrofits (insulation, heat pumps, HVAC upgrades) cut that, but they are expensive, so organisations must choose.
- Raw energy use misleads: a hospital always uses more than a warehouse, and a building in a cold state always heats more.

What we need: how much a building should use, given its type, size, age and weather.

Our approach: actual vs expected energy

1. Predict

A model learns each building's site EUI from its type, size, age, state and weather.

2. Benchmark

A second model, without the Energy Star rating, gives each building's expected EUI.

3. Flag and rank

Buildings using 1.5× their expected EUI or more become retrofit candidates, ranked by wasted MWh and CO_2.

Site EUI (energy use intensity) = energy used per square foot per year, in kBtu/ft²/yr. Dividing by floor area means big buildings are not flagged just for being big.

The data: WiDS Datathon 2022

75,757
building-years to train on (Years 1–6, 64 columns)

9,705
buildings to predict (Year 7, no answers given)

56,382
distinct buildings: the same building repeats across years

Cleaning step	Why
Dropped wind and fog columns	54–60% empty in train, 88–94% in test
Removed 12 duplicate rows; build years before 1800 set to missing (25)	Data-entry errors
Filled missing Energy Star and year built with the facility-type median, plus a “missing” flag	Learned from training rows only, so nothing leaks from the test year
36 monthly temperatures → winter minimum and summer maximum	Near-duplicates of each other and of degree days
Modelled log(EUI) instead of EUI	Long right tail: a few buildings exceed 900 kBtu/ft²/yr

How we tested: two traps, one rule

Trap 1: State 6

State 6 is 67% of the training rows but 0% of the test year (Year 7). So every score is also reported with State 6 removed.

Trap 2: memorisation

The same building repeats across years. Buildings the model has seen: RMSE 29.7. New buildings: RMSE 45.6. A random split would flatter the model.

Our rule: tune on Years 1–5, score once on Year 6 (never used for any decision), then retrain on Years 1–6 to predict Year 7. Boosting was tuned with time-series cross-validation: train on Years 1–3 → check Year 4; train on Years 1–4 → check Year 5.

RMSE (root mean squared error): the typical size of a prediction miss, in kBtu/ft²/yr. Lower is better.

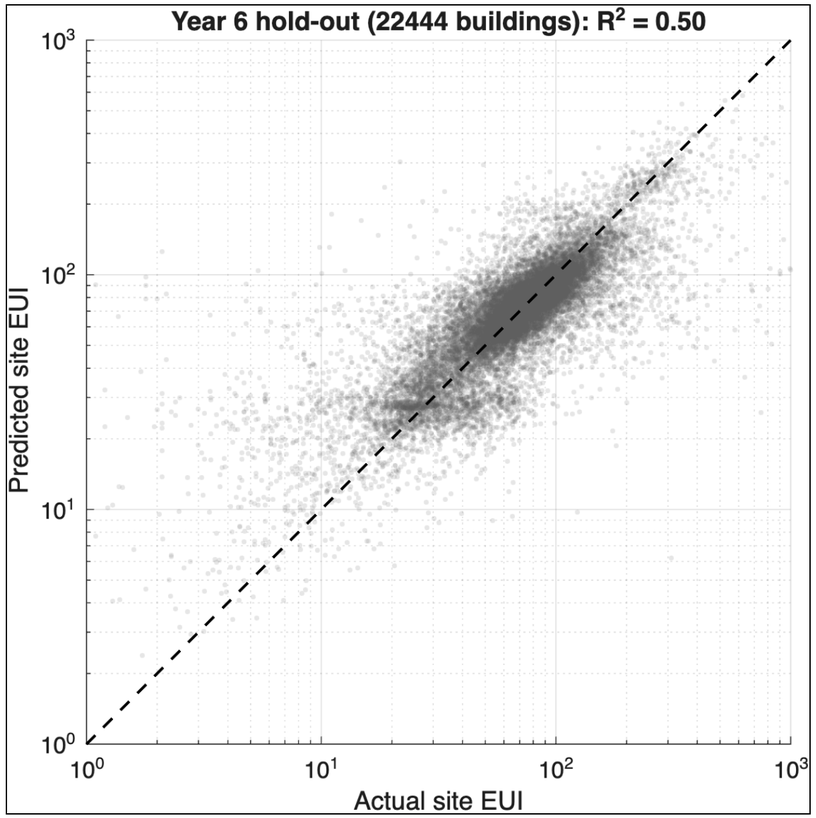
Eight models, all scored on Year 6

#	Model	RMSE	MAE	R^2	RMSE without State 6
0	Facility-type median (baseline)	50.7	27.2	0.21	53.6
1	Linear regression	44.6	23.1	0.39	49.1
2	Regression tree	42.7	21.5	0.44	46.8
3	Bagged trees (100)	42.6	22.7	0.44	44.8
4	Boosted trees, raw target	44.4	22.3	0.39	40.2
5	Boosted trees, log target	40.2	18.8	0.50	42.4
6	Neural network	53.9	29.6	0.11	61.3
7	Tuned boosted trees (final model)	40.3	18.7	0.50	43.2

Biggest gain: modelling log(EUI), model 4 → 5. Tuning confirmed those settings (40.3 vs 40.2).

MAE: average absolute error. R^2: share of the differences between buildings that the model explains (1 = perfect).

Final model: 20% better than baseline



40.3
RMSE on Year 6
(baseline 50.7)

0.50
R² on Year 6
(baseline 0.21)

Model: gradient-boosted trees (LSBoost) on log(EUI): 298 trees, up to 60 splits each, learning rate 0.05, chosen by time-series cross-validation.

Strongest drivers: facility type, Energy Star rating, floor area and year built.

Reading the chart: typical buildings sit on the diagonal; the big misses are extreme users, which the retrofit engine examines next.

Retrofit engine: flag and rank

Benchmark model: same method, but without the Energy Star rating (it already measures efficiency, so it would excuse inefficient buildings), and trained without any Year 6 building, so it never judges a building it has seen.

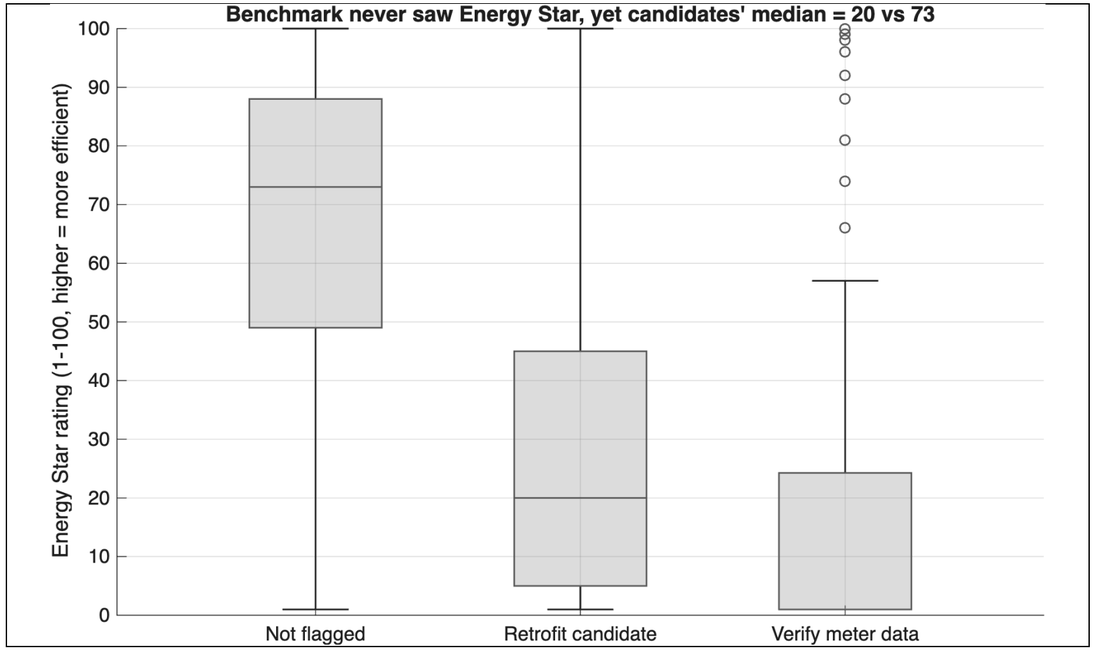
Ratio = actual EUI ÷ expected EUI

Excess energy (MWh/yr) = (actual − expected EUI) × floor area × 0.000293

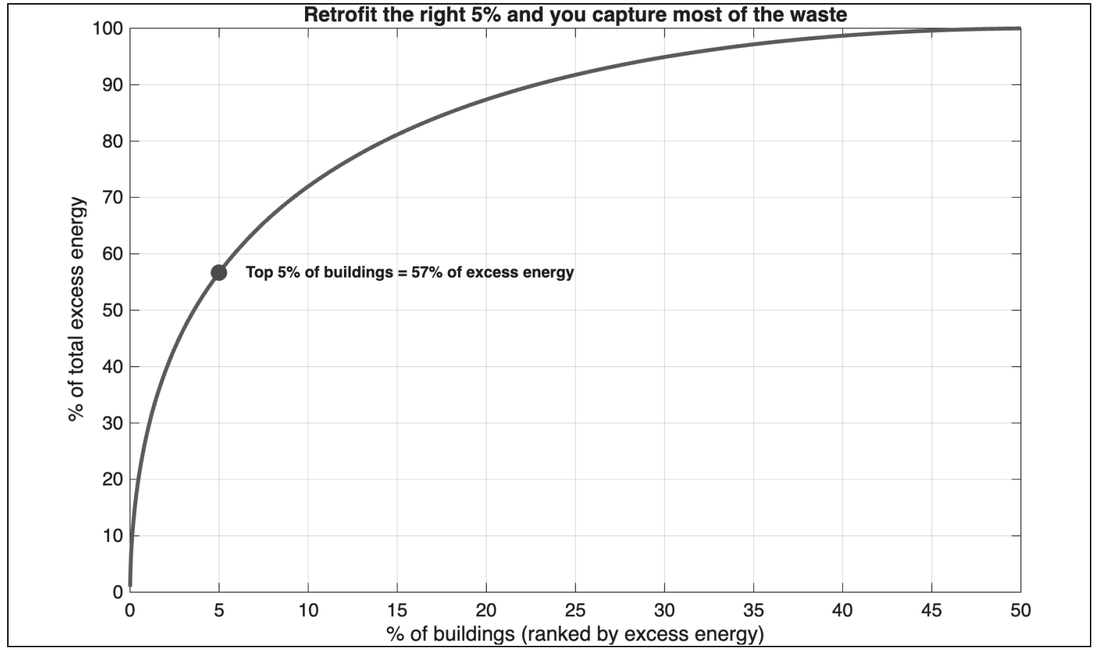
Excess CO_2 (t/yr) = excess MWh × 0.38 (assumed grid average)

Actual ÷ expected	**Outcome**	**Year 6 buildings**
Below 1.5×	Not flagged	19,364
1.5× to 5×	Retrofit candidate	2,973 (13.2%)
5× or more	Verify meter data (likely error)	107

Checking the flags: two results

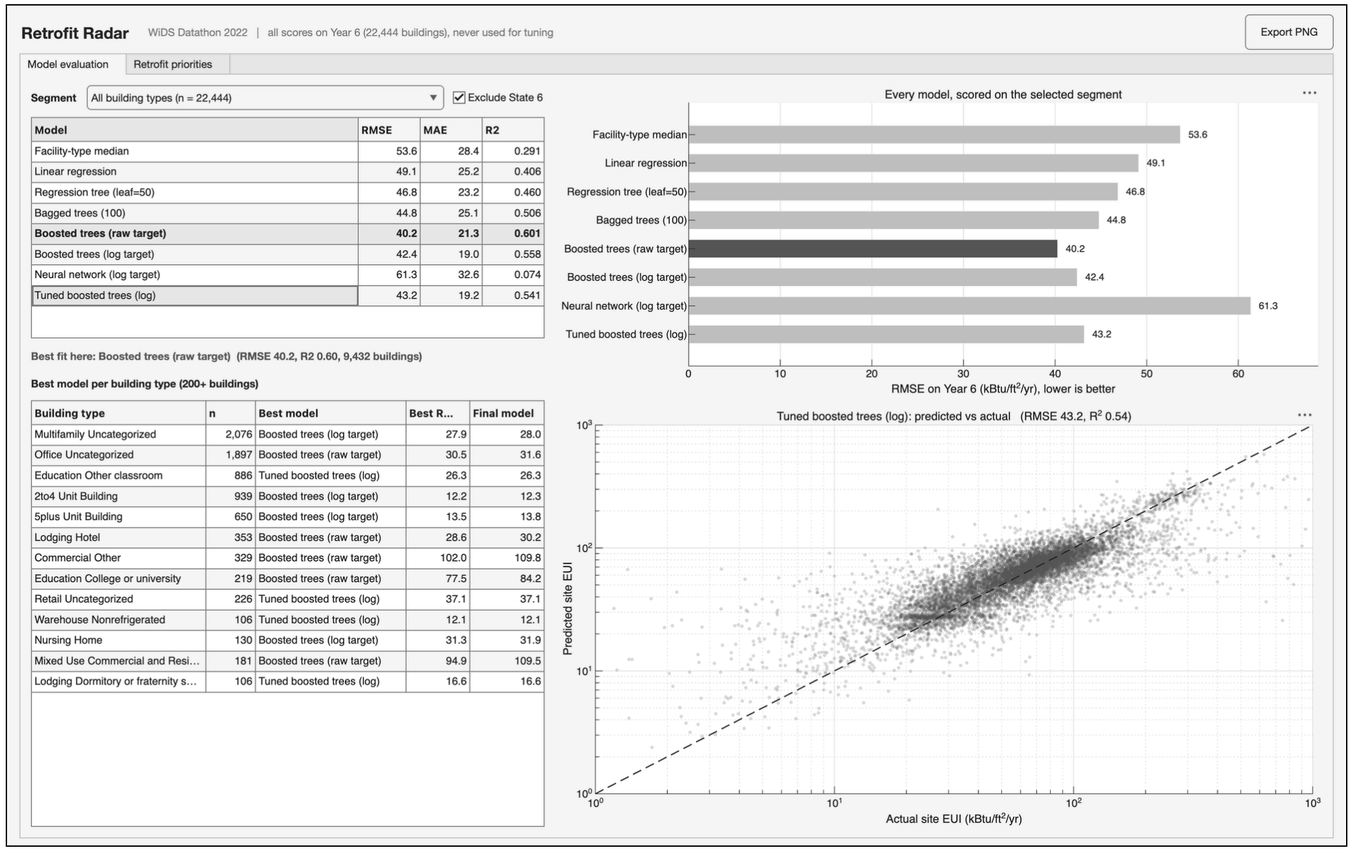


Independent check: the benchmark never saw Energy Star ratings, yet candidates have a median rating of 20, vs 73 for unflagged buildings.



Concentration: the top 5% of buildings hold 57% of all excess energy, so a small budget aimed at the top of the list captures most of it.

The MATLAB app: test the models yourself



Model evaluation

Re-scores all 8 models on any building type, with or without State 6, and highlights the best fit.

Retrofit priorities

Filters the ranked list by building type and ratio. Click a building to see its actual vs expected energy.

Finding: outside State 6, raw-target boosting is best (RMSE 40.2 vs 43.2 for our final model). We report this rather than switch after seeing Year 6.

Impact: a ranked list of buildings to audit

2,973

retrofit candidates in Year 6 (13% of buildings), ranked in a CSV list

8,215

GWh a year of excess energy if every candidate matched its benchmark (an upper bound)

3,122

kilotonnes of CO_2 a year, at an assumed 0.38 t CO_2 per MWh

Also delivered: energy predictions for all 9,705 Year 7 buildings, 107 readings flagged for meter checks, and a median excess of about 1,305 MWh a year per candidate.

Limits, assumptions and next steps

Limits and key assumptions

- A screening tool, not an audit: a flag says which building to check, not what to fix.
- R^2 0.50: the data has no occupancy, opening hours or equipment information.
- CO_2 uses one assumed factor (0.38 t/MWh); savings are an upper bound.
- Years and states are anonymised, and State 6 skews the overall scores.

Next steps

1. Choose models per building type, or use the raw-target model for Year 7 (no State 6).
2. Add retrofit costs to rank by cost per tonne of CO_2 saved.
3. Re-run each year as new meter data arrives.
4. Add occupancy and operating-hours data where available.

Full report, code and app: github.com/chuaguanyu0718-dotcom/building-energy-prediction